In this exercise, you'll build and deploy a secure chatbot user interface using AWS services. The frontend will be hosted on S3 and served through CloudFront with proper security configurations.

What You'll Learn

By the end of this exercise, you will have:

- Built a responsive chatbot UI with HTML, CSS, and JavaScript
- Deployed the frontend to AWS S3 with secure access controls
- Configured CloudFront for global content delivery
- Implemented proper security practices for web hosting
- Created a foundation for backend integration

Step 1: Build the Chatbot Frontend

Using your existing **MyVibeChatbot** project from the previous exercise, instruct Kiro to build the chatbot interface.

```
Build a simple chatbot UI with the following requirements:
   - I want the files to be hosted on S3 and served by CloudFront.
   - For security reasons, I do not want public access to my S3 bucket.
   - Also, do not enable versioning for the S3 bucket.
   - The S3 bucket policy must allow the workshop participant role to upload files.

The chatbot UI interacts with the backend with POST requests.
We will build the backend separately in later steps.
When the user enters a message, the chatbot UI does not send the POST request, instead simply respond with an ACK.

We will build the UI using CloudFormation template with the following requirements:
   - Stack name MUST start with "kiro-" prefix (e.g., "kiro-chatbot-ui-stack")
   - Use the tag "Project=kiro-workshop" for all AWS resources that can be tagged

Just create all the required files. Do not deploy for now.
```

Development Approach
Think of yourself as a Product Manager and Kiro as your development team. You define the requirements, Kiro implements the solution, and you provide feedback and guidance throughout the process.

Step 2: Validate CloudFormation Template

Before deploying infrastructure, it's important to validate the CloudFormation template against AWS best practices and prevent post-deployment access issues.

```
Before deployment, validate the CloudFormation template:
1. Check against AWS documentation for best practices
2. Verify the S3 bucket policy includes participant role permissions for uploads

If the participant permissions are missing, add them now before deployment. Ensure the policy has separate statements for
1. s3:ListBucket (on bucket)
2. s3:PutObject/DeleteObject/GetObject (on objects)
```

Why This Matters
The S3 bucket policy must allow both CloudFront (for serving content) and the participant role (for uploading files). Without participant permissions, you'll get Access Denied errors when trying to upload files after deployment.

Expected Architecture

Your deployment should include:

- **S3 Bucket**: Private bucket for hosting static files
- **CloudFront Distribution**: CDN for global content delivery
- **Origin Access Control (OAC)**: Secure access from CloudFront to S3
- **Bucket Policy**: Allows CloudFront OAC AND participant role access

---

Step 3: Deploy the ChatBot Frontend

Once the CloudFront stack is checked successfully, instruct Kiro to deploy it.

```
Now deploy the CloudFormation stack for me.
```

Kiro uses the AWS credentials you set before to deploy the CloudFormation stack to an AWS account. Open the AWS account, choose the right region, and find the CloudFormation stack. The deployment may take some time to finish. However, if it takes too long, check the CloudFormation stack to find out what is blocking the deployment. You may need to instruct Kiro to fix any errors and deploy again.

Let Kiro Handle Commands

When Kiro suggests running terminal commands or AWS CLI operations, ask Kiro to execute them for you instead of running them manually.

```
Can you run that command for me? I'd prefer if you handle the deployment steps.
```

Command Execution
Always ask Kiro to run commands rather than executing them yourself. This ensures proper integration with your development environment and maintains consistency.

Step 4: Test and Iterate

You should be able to find an url like "https://xxxx.cloudfront.net" from output. Paste the url in a browser to play with your chatbot. The chatbot should reply with something like "ACK" when you input any text.

You also should be able to find an updated architecture diagram. If not, click on the agent hook. Check whether it is running, or whether it is waiting your permission to run any commands.

Once you confirmed both, work with Kiro iteratively to refine the chatbot UI:

1. **Review the initial deployment**
2. **Test the chatbot interface**
3. **Provide feedback for improvements**
4. **Iterate until you have a working solution**

Iterative Development
You'll see Kiro making mistakes and fixing them automatically. This is normal and part of the development process. Provide guidance and feedback to help Kiro arrive at the optimal solution.

Expected Results

After completing this exercise, you should have:

- ✅ A responsive chatbot UI with modern design
- ✅ Secure S3 bucket hosting (no public access)
- ✅ CloudFront distribution for content delivery
- ✅ Proper security configurations and policies
- ✅ Working chatbot interface that responds with ACK messages
- ✅ Foundation ready for backend integration

Troubleshooting Common Issues

▶ **S3 Public Access Blocked**

▶ **Deployment Stuck. Bucket Policy Issues**

▶ **CloudFront Cache Issues**

:::

▶ **Cannot Upload Files to S3 (Access Denied)**

Verification: After updating the stack, test file upload:

```
echo "test" > test.txt && aws s3 cp test.txt s3://YOUR-BUCKET-NAME/test.txt && aws s3 rm s3://YOUR-BUCKET-NAME/test.txt && rm test.txt
```

:::

▶ **Resource Naming Conflicts**

Architecture Overview

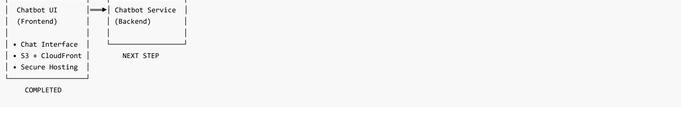
▶ **View Deployment Architecture**

This exercise establishes the foundation for a production-ready chatbot application with proper security controls and scalable architecture.

Next Steps

With your secure chatbot frontend deployed, you're ready to build the backend services in the next exercise. You'll work with Kiro to create a serverless backend API using AWS Lambda and API Gateway that will handle chat requests from your frontend and provide the foundation for adding AI intelligence later.

Workshop Progress



Let's Create A Chatbot Service
